

ses this will complicate the process of making schema changes to Postgres and Elasticsearch. Today we already need to be careful to make changes comply with our zero-downtime deployments. For example, when removing a column we need to make changes over 3 separate deployments. We have tooling like postmigrate that helps with these kinds of changes to reduce the number of merge requests needed, but these will be complicated when we are dealing with deploying multiple Rails applications that will be at different versions at any one time. This problem will be particularly tricky to solve for shared databases like our plan to share the users related tables among all Cells.

A key benefit of Cells may be that it allows us to run different customers on different versions of GitLab.

We may choose to update our own Cell before all our customers giving us even more flexibility than our current canary architecture.

But doing this means that schema changes need to have even more versions of backward compatibility support which could slow down development as we need extra steps to make schema changes.

1. Definition

2. Data flow

3. Proposal

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/secrets.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

Where possible, each Cell should have its own distinct set of secrets.

However, there will be some secrets that will be required to be the same for all Cells in the cluster.

1. Definition

GitLab has a few secrets that need to be configured.

These secrets are stored in different locations depending on the installation method.

- Secrets for encryption: secretkeybase (session data) and dbkeybase (encryption at rest in the database).
- Secrets for features: otpkeybase and openidconnectsigningkey.
- Secret for encrypted files (similar to the Rails credentials feature): encryptedsettingskeybase.

In addition to secrets stored in the "secrets" file, GitLab also uses specific secret files for inter-component communication, for example, GitLab Shell secret, and used only within a Cell.

Last but not least, many features need credentials to be set up. These credentials are stored in different locations depending on the installation method:

- /etc/gitlab/gitlab.rb for Omnibus
- Kubernetes secrets for Charts
- config/gitlab.yml for source installation

2. Data flow

3. Proposal

1. secretkeybase is used for: encrypted cookies, signed cookies, and Active Storage files, based on <https://guides.rubyonrails.org/security.html#session-storage>.

We don't use Active Storage, as far as I know, so it's only cookies that are affected. Given that for Cells 1.0, users will be tied to a single cell, and since

the session cookie will include the cell ID in its name, it's fine to have a unique secretkeybase per Cell.

Investigation issue: secretkeybase.

1. dbkeybase is used for encryption at rest in the database, and will need to be consistent across all Cells, so that

data can be moved easily between Cells.

Investigation issue: dbkeybase.

1. This is especially true for the dbkeybase secret which is used for encrypting data at rest in the database - so that Projects that are transferred to another Cell will continue to work. We do not want to have to re-encrypt such rows when we move Projects/Groups between Cells.

1. In the future, we might support rotating such secrets, so that if the Org mover needs to move an organization

from Cell1 to Cell2, it would decrypt data with Cell1 key and then re-encrypt it with Cell2 key. This is out of scope for Cells 1.0.

1. Secrets for features (otpkeybase and openidconnectsigningkey) will need to be consistent across all

Cells, so that the UX is consistent.

Investigation issues: otpkeybase, openidconnectsigningkey.

1. Since we'll support cluster-wide Application Settings, we should first migrate these secrets to

ApplicationSetting attributes first, so that the consistency of their value on each Cell is handled by

the Application Settings synchronization mechanism.

1. Secrets for encrypted files can be different on each Cell, as long as the credentials inside the encrypted files

are the same. That said, it might be simpler to also use the same encryptedsettingskeybase on all Cells for

simplicity sake. Investigation issue.

1. Secrets for inter-component communication inside a Cell only should be uniquely generated per Cell.

1. In general, credentials for various features should be the same on all Cells, unless a feature is set up in a

Cell-specific way (e.g. if LDAP needs to be set up on a specific Cell).

GitLab Dedicated already have a solution for

sharing SMTP credentials in a multi-tenant environment

so we should use a similar solution to set up cluster-wide configuration.

4. Evaluation

4.1. Pros

- Secrets are consistent accross most cells
- No need to handle data migration between cells

4.2. Cons

- Security of cells is not higher than the current GitLab.com installation

SECTION: engineering/architecture/design-documents/cells/impacted_features/snippets.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement.

This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

Snippets will be scoped to an Organization. Initially it will not be possible to aggregate snippet collections across Organizations. See also issue 416954.

1. Definition

Two different types of snippets exist:

- Project snippets. These snippets have URLs like `/<group>/<project>/-snippets/123`
- Personal snippets. These snippets have URLs like `/-snippets/123`

Snippets are backed by a Git repository.

2. Data flow

3. Proposal

3.1. Scoped to an organization

Both project and personal snippets will be scoped to an Organization.

- Project snippets URLs will remain unchanged, as the URLs are routable.
- Personal snippets URLs will need to change to be
/-/organizations/<organization>/snippets/123,
so that the URL is routeable

Creation of snippets will also be scoped to a User's current Organization. Because of that, we recommend renaming personal snippets to organization snippets once the Organization is rolled out. A User can create many independent snippet collections across multiple Organizations.

4. Evaluation

Snippets are scoped to an Organization because Gitaly is confined to a Cell.

4.1. Pros

- No need to have clusterwide Gitaly.

4.2. Cons

- We will break snippet discovery.
- Snippet access may become subordinate to the visibility of the Organization.

SECTION: engineering/architecture/design-documents/cells/impacted_features/template.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

TL;DR

1. Definition

2. Data flow

3. Proposal

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/uploads.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

TL;DR

1. Definition

2. Data flow

3. Proposal

4. Evaluation

4.1. Pros

4.2. Cons

SECTION: engineering/architecture/design-documents/cells/impacted_features/user-profile.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

The existing User Profiles will initially be scoped to an Organization. Long-term, we should consider aggregating parts of the User activity across Organizations to enable Users a global

view of their contributions.

1. Definition

Each GitLab account has a User Profile, which contains information about the User and their GitLab activity.

2. Data flow

3. Proposal

User Profiles will be scoped to an Organization. We follow the same pattern as is used for Your Work, meaning that profiles are always seen in the context of an Organization.

- User Profile URLs will reference the Organization with the following URL structure `/-/organizations/<organization>/username`.
- Users can set a Home Organization as their main Organization.
- The default User Profile URL `/<username>` will refer to the user's Home Organization, or the default Organization if the user's Home Organization is not set.
- Users who do not exist in the database at all display a 404 not found error when trying to access their User Profile.
- User who haven't contributed to an Organization display their User Profile with an empty state.
- When displaying a User Profile empty state, if the profile has a Home Organization set to another Organization, we display a call-to-action allowing navigation to the main Organization.
- Breadcrumbs on the User Profile will present as `[Organization Name] / [Username]`.

See issue 411931 for design proposals.

4. Evaluation

We expect the majority of Users to perform most of their activity in one single Organization. This is why we deem it acceptable to scope the User Profile to an Organization at first. More discovery is necessary to understand which aspects of the current User Profile are relevant to showcase contributions in a global context.

4.1. Pros

- Viewing a User Profile scoped to an Organization allows you to focus on contributions that are most relevant to your Organization, filtering out the User's other activities.
- Existing User Profile URLs do not break.

4.2. Cons

- Users will lose the ability to display their entire activity, which may lessen the effectiveness of using their User Profile as a resume of achievements when working across multiple Organizations.

SECTION: engineering/architecture/design-documents/cells/impacted_features/your-work.md

{{% alert %}}

This document is a work-in-progress and represents a very early state of the Cells design. Significant aspects are not documented, though we expect to add them in the future. This is one possible architecture for Cells, and we intend to contrast this with alternatives before deciding which approach to implement. This documentation will be kept even if we decide not to implement this so that we can document the reasons for not choosing this approach.

{{% /alert %}}

Your Work will be scoped to an Organization.

Counts presented in the individual dashboards will relate to the selected Organization.

1. Definition

When accessing gitlab.com/dashboard/, users can find a focused view of items that they have access to.

This overview contains dashboards relating to:

- Projects
- Groups
- Issues
- Merge requests
- To-Do list
- Milestones
- Snippets
- Activity
- Workspaces
- Environments
- Operations
- Security

2. Data flow

3. Proposal

Your Work will be scoped to an Organization, giving the user an overview of all the items they can access in the Organization they are currently viewing.

- Issue, Merge request and To-Do list counts will refer to the selected Organization.
- The URL will reference the Organization with the following URL structure `/-/organizations/<organization>/dashboard`.
- The default URL `/dashboard` will refer to the Home Organization.

4. Evaluation

Scoping Your Work to an Organization makes sense in the context of the proposed Organization navigation.

Considering that we expect most users to work in a single Organization, we deem this impact

acceptable.

4.1. Pros

- Viewing Your Work scoped to an Organization allows Users to focus on content that is most relevant to their currently selected Organization.

4.2. Cons

- Users working across multiple Organizations will have to go to each Organization to access all of their work items.

SECTION: engineering/architecture/design-documents/cells/infrastructure/_index.md

{{< engineering/design-document-header >}}

Pre-reads

1. Cells Iteration, specifically Cells 1.0
1. GitLab Dedicated
1. GitLab Dedicated Architecture

Philosophy

- Cell-local by default: All services should be cell-local, and not global, unless there are documented, and good reasons why they aren't cell-local.

If we keep things cell-local, communication between the cell and service stays internal, the service has to run at a smaller scale, and the blast radius is much smaller.

Example, Gitaly and GitLab Registry are cell-local.

- Homogeneous environments: For now, every GitLab cell should look the same. Bootstrapping and provisioning should be done in an automated way.

For the first iteration all Cells are the same size, there are benefits of running different sizes but this adds complexity and scope.

- Fresh start, but not so much: Brand new GitLab instances are created, so it's tempting to redo everything. We have to balance the existing infrastructure, dedicated tooling, and time.

- All operations get rolled out the same: Configuration changes, Feature Flags, Deployments, and operational tasks ideally go through the same process of rolling out a change.

Having 1 way of doing things can bring efficiencies and a single source of truth for automation.

- Centralize Tooling: We have a lot of tooling to manage GitLab.com and separate tooling for GitLab Dedicated,

which creates silos, duplication of effort, and less portability.

We have to provision multiple Cells for GitLab.com, we need new tooling, GitLab Dedicated built tooling just for this reason.

We should try to use this tooling as much as possible, if there are things we don't agree with we should try disagree, commit, and disagree to improve a single tool.

It is ok to start with tooling that has shortcomings, an iterative approach leads to one mature product instead of two.

Glossary/Ubiquitous Language

Ubiquitous Language

- Provision: When we create a new Cell. Example: We provisioned Cell 5, which is a brand new Cell.
- Deploy: When we change the running code inside of an existing Cell. Example: We deployed the new auto-deploy version on GitLab.com.
 - Blueprint
- Configuration change: When we change any configuration on the application or infrastructure. Example: We did a configuration change on labels added to VMs.
- Tenant: A GitLab instance provisioned through GitLab Dedicated tooling (Instrumentor). A Tenant can be either a GitLab Dedicated Customer Instance, or a Cell Instance.
- Cell: A Tenant provisioned to be a part of GitLab.com where multiple customers are served through a single Tenant.
- Legacy Cell: The existing GitLab.com deployment.
- Ring: A collection of Cells grouped as single deployment stage target. Example: Cells in Ring 2 will deploy changes after Cells in Ring 1.
- Cluster: A collection of Cells, and the existing GitLab.com infrastructure. Example: We need to change the version of Registry in the Cluster.
- Fleet: The collection of all SaaS environments, both single-tenant and multi-tenant, that collectively form our production environments.
 - This includes existing GitLab.com infrastructure, Cells, and Dedicated.

Architecture

Below is the Cell architecture. You can find the current GitLab.com architecture (pre-Cells) in <https://handbook.gitlab.com/handbook/engineering/infrastructure/production/architecture/>.

```
plantuml
@startuml
!include <k8s/Common>
!include <k8s/Context>
!include <k8s/Simplified>
!include <k8s/OSS/all>
!include <gcp/GCPCommon>
!include <gcp/Compute/all>
!include <gcp/Databases/all>
!include <gcp/Security/all>
!include <gcp/Storage/all>

skinparam actorStyle awesome
skinparam frame {
    borderColor<<gcpproject>> 4285F4
}
skinparam frame {
    borderColor<<vpc>> F4B400
}
skinparam frame {
    borderColor<<gcp>> 0F9D58
```

```

}
skinparam frame {
  borderColor<<cluster>> DB4437
}
skinparam cloud {
  borderColor<<cloudflare>> F48120
}

:User:
cloud gitlab.com <<cloudflare>> {
  [DoS Protection]-->[WAF]
  [WAF]-->[RoutingService]
}
cloud "cloud.gitlab.com" <<cloudflare>> {
  [Cloud Connector]-->[AI Gateway]
}
:User:->gitlab.com
:User:->cloud.gitlab.com

cloud "ClickHouse Cloud" {
  database "ClickHouse" as cell1Clickhouse
  database "ClickHouse" as cell2Clickhouse
}

frame "Google Cloud Platform" <<gcp>> {
  frame "topology-service" <<gcpproject>>{
    rectangle "TopologyService" as TopologyService
  }
  frame "Cell Cluster" <<cluster>> {
    frame "gitlab-production" <<gcpproject>>{
      frame "gprd (Shared VPC Network)" <<vpc>> as gprdVPC {
        rectangle "Frontend" as primaryFrontend {
          node "HAProxy"
        }

        rectangle "Compute" as primaryCompute {
          node zonal [
            <b> zonal cluster x3
            ===
            api
            ---
            web
            ---
            git
            ---
            gitlab-shell
            ---
            websockets
            ---
            registry

```

```

]

node regional [
  <b>regional x1
  ===
  sidekiq
  ---
  kas
  ---
  zoekt
  ---
  cny
  ---
  pages
]
}

rectangle "Storage" as primaryStorage {
  database "patroni-main"
  database "patroni-ci"
  database "patroni-registry"
  database "redis (multiples)"
  file "object storage" as primaryObjectStorage
}

primaryFrontend <--> primaryCompute
primaryCompute <--> primaryStorage
}
}

frame "gitlab-ci" <<gcpproject>> {
  node "runner managers" as runnerManager

  runnerManager --> "HAProxy"
}
frame "gitlab-ci-" <<gcpproject>> {
  node "ephemeral VMs"
}
runnerManager --> "gitlab-ci-"

frame "gitlab-gitaly-gprd-" <<gcpproject>> {
  file "gitaly-[1,9]" as primaryGitaly
}
primaryCompute <--> primaryGitaly
primaryGitaly .r[F4B400]. gprdVPC

frame "cell-01HV3FBWXHSYBAR2R2" as cell1 <<gcpproject>> {
  frame "cell-01HV3FBWXHSYBAR2R2 VPC" <<vpc>> as cell1VPC {
    ClusterBoundary(cell1gke, "cell1gke") {
      NamespaceBoundary(cell1gitlab, "gitlab") {

```

```

    KubernetesPod(cell1web, "Web", "")
    KubernetesPod(cell1shell, "Shell", "")
    KubernetesPod(cell1registry, "Registry", "")
    KubernetesPod(cell1sidekiq, "Sidekiq", "")
  }
  NamespaceBoundary(cell1Observability, "monitoring") {
    KubernetesPod(cell1Prom, "Prometheus", "")
  }
}

storage "Storage" as cell1Storage {
  CloudSQL(cell1Postgres, "PostgreSQL", "")
  CloudMemorystore(cell1Redis, "Redis", "")
  CloudStorage(cell1ObjectStorage, "GCS", "")
  KeyManagementService(cell1SecretStorage, "GKMS", "")
  ComputeEngine(cell1Gitaly, "Gitaly", "")
}
cell1gke <--> cell1Storage
cell1gke <---> cell1Clickhouse
}

frame "cell-01HV3FBYX37C18SW05" as cell2 <<gcpproject>> {
  frame "cell-01HV3FBYX37C18SW05 VPC" <<vpc>> as cell2VPC {
    ClusterBoundary(cell2gke, "cell2gke") {
      NamespaceBoundary(cell2gitlab, "gitlab") {
        KubernetesPod(cell2web, "Web", "")
        KubernetesPod(cell2shell, "Shell", "")
        KubernetesPod(cell2registry, "Registry", "")
        KubernetesPod(cell2sidekiq, "Sidekiq", "")
      }
      NamespaceBoundary(cell2Observability, "monitoring") {
        KubernetesPod(cell2Prom, "Prometheus", "")
      }
    }
  }

  storage "Storage" as cell2Storage {
    CloudSQL(cell2Postgres, "PostgreSQL", "")
    CloudMemorystore(cell2Redis, "Redis", "")
    CloudStorage(cell2ObjectStorage, "GCS", "")
    KeyManagementService(cell2SecretStorage, "GKMS", "")
    ComputeEngine(cell2Gitaly, "Gitaly", "")
  }

  cell2gke <--> cell2Storage
  cell2gke <---> cell2Clickhouse
}

cell1VPC <. gprdVPC : Private Service Connect

```

```
    cell2VPC <. gprdVPC : Private Service Connect
  }
```

```
  "Cell Cluster" -u-> cloud.gitlab.com
}
[RoutingService]-[thickness=1]->TopologyService
[cell1gke]-[thickness=1]->TopologyService
[cell2gke]-[thickness=1]->TopologyService
[primaryFrontend]-[thickness=1]->TopologyService
[RoutingService]-[thickness=3]->primaryFrontend
[RoutingService]-[thickness=3]->cell1gke
[RoutingService]-[thickness=3]->cell2gke
@enduml
```

KAS

```
plantuml
@startuml
```

```
skinparam frame {
  borderColor<<customer>> F4B400
}
skinparam frame {
  borderColor<<gcp>> 4285F4
}
skinparam cloud {
  borderColor<<cloudflare>> F48120
}

together {
  frame "cluster 1" <<customer>> {
    component "agentk" as cluster1AgentK
  }

  frame "cluster 2" <<customer>> {
    component "agentk" as cluster2AgentK
  }

  frame "cluster 3" <<customer>> {
    component "agentk" as cluster3AgentK
  }

  frame "workstation" <<customer>> {
    component "kubectl"
  }
}
```

```
cloud wss://kas.gitlab.com <<cloudflare>> as kas.gitlab.com {
```

```

    component "routing service"
  }

cluster1AgentK <..d..> kas.gitlab.com
cluster2AgentK <..d..> kas.gitlab.com
cluster3AgentK <--d--> kas.gitlab.com
kubectl <--d--> kas.gitlab.com

together {
  frame "gprd-gitlab-cell-1" <<gcp>> {
    component kas as kasCell1
    component webservice as webserviceCell1
    component redis as redisCell1
    collections "gitaly(s)" as gitalyCell1

    kasCell1 <-d-> webserviceCell1
    kasCell1 <-d-> redisCell1
    kasCell1 <-d-> gitalyCell1
  }

  frame "gprd-gitlab-cell-2" <<gcp>> {
    component kas as kasCell2
    component webservice as webserviceCell2
    component redis as redisCell2
    collections "gitaly(s)" as gitalyCell2

    kasCell2 <-d-> webserviceCell2
    kasCell2 <-d-> redisCell2
    kasCell2 <-d-> gitalyCell2
  }
}

"routing service" <--d--> kasCell1
"routing service" <--d--> kasCell1
"routing service" <..d..> kasCell2
"routing service" <..d..> kasCell2

@enduml

```

Rings

Rings serve as the basis of the mental model of how we group the Cells we provision and the existing infrastructure.

Inside of a ring, there is X number of Cells, subsequent rings consist of more cells gradually covering the entire fleet.

Each Ring will be a superset of the previous rings.

For example ring zero only contains the Cells in ring zero,
ring 5 contains the Cells in ring 5 and all other rings before it.

Changes cascade outwards from inner rings to outer rings in discrete stages.

For example If a change has reached ring 5, it will have reached ring 4, 3, 2, and 1.

Any type of rollout will allow start from ring 0 and move to subsequent rings if the change is successful,

if it fails we can stop the rollout and we don't affect all of our customers.

With this type of progressive rollout, we'll get the following benefits:

1. Changes have a smaller blast radius, not affecting all customers at once.
1. Clear boundaries on how to roll out a change.
1. Removes the need of having different environments like Staging, all Cells will be production.
1. The more confident we are with a change the wider the audience.

```
plantuml
@startuml
```

```
skinparam frame {
    borderColor<<Cells 1.0>> 0F9D58
}
```

```
skinparam frame {
    borderColor<<Cells 1.5+>> F4B400
}
```

left to right direction

```
frame "Ring 3" <<cells 1.5+>> {
    component "01HWRY6Y73W6TW3BHC"
    component "01HWRY6Y740CZSBCGB"
    component "01HWRY6Y74AHZ0AFZ1"
    component "01HWRY6Y743018H2N7"
```

```
frame "Ring 2" <<Cells 1.0>> {
    component "01HWRY6Y74HM0SKTZ9"
    component "01HWRY6Y74YZF5DX5C"
    component "01HWRY6Y74QYG7VV2Y"
    component "01HWRY6Y745BE459Y6"
    component "01HWRY6Y74915AAS8E"
    component "01HWRY6Y74EV3ZVXF5"
    component "01HWRY6Y74REVKJK3P"
    component "01HWRY6Y748ZBGR9G1"
    component "01HWRY6Y74AR5HHJ4H"
    component "01HWRY6Y743V5085XK"
```

```
frame "Ring 1" <<Cells 1.0>> {
    frame "Ring 0" <<Cells 1.0>> {
        component "Canary stage" <<legacy>> as cry
        component "01HWRY6Y745RS405F6"
    }
}
```

```

    component "Main stage\nLegacy Cell" <<legacy>> as Legacy
  }
}
}

@enduml

```

For Cells 1.0 our aim is to have up to 10 cells inside ring 2.
 The number of Cells in a ring is arbitrary, their size is still to be determined.
 It will take into consideration our necessity to adequately test auto-deploy packages before a public release,
 the speed of a full production rollout for security fixes,
 and the protection from outages or bugs of our users.

Where we'll eventually use rings for managing all the changes in the Cells environments.

Staging

We do not have the traditional Staging environment in rings,
 because we can test changes in the first rings which achieves the same outcome.
 This doesn't mean that we will shut off the existing staging environment,
 which will still be in use for the non-cell infrastructure.

With this set up we'll end up removing some of the problems we have with staging right now:

1. Staging is not a real representation of Production.
1. We consider Staging as Production because it blocks deployments.
1. The configuration of Staging can drift from Production.

Large Domains

The infrastructure is multifaceted and all teams have a role in setting up the cell infrastructure.

The Confidence column refers to how confident we are with the specific domain and its path forward for Cells.

When we have a blueprint merged ideally the confidence should move to · because we have a blueprint that provides direction to that domain.

Domain	Owner	Blueprint
Confidence		
-----	-----	-----
-----	-----	-----
Routing	group::tenant scale	Blueprint
·		
Cell Control Plane	group::Delivery/team::Foundations	To-Do
·		
Cell Sizing	team::Scalability-Observability	To-Do
		·
CI Runners	team::Scalability-Practices	Blueprint


```

| ·      |
| Databases          | team::Database Reliability    | Blueprint
| ·      |
| Deployments        | group::Delivery              | Blueprint
| ·      |
| Observability       | team::Scalability-Observability | Blueprint
| ·      |
| Cell Architecture and Tooling | team::Foundations          | Blueprint
| ·      |
| Provisioning         | team::Foundations          | To-Do
| ·      |
| Configuration Management/Rollout | team::Foundations          | To-Do
| ·      |
| Disaster Recovery    | team::Production Engineering | Blueprint
| ·      |

```

```

plantuml
@startuml
skinparam component {
    BackgroundColor White
    BorderColor Black
}

```

```

rectangle "Domains as Downstream Dependencies" line.dashed {
    component "Control Plane"
    component "Cell Sizing"
    component "Databases"
    component "Routing"
    component "CI Runners"
    component "Cell Architecture and tooling"
}

```

```

component "Deployments"
component "Observability"
component "Provisioning"
component "Configuration Management"

```

```

"Deployments" -d-> "Control Plane" : Ring Definition
"Deployments" -d-> "Provisioning": Cell needs to exist to deploy
"Configuration Management" -d-> "Control Plane": Ring Definition
"Provisioning" -d-> "Cell Sizing": Size we are going to Provision
"Provisioning" -d-> "Databases": Database to provision
"Provisioning" -d-> "Observability": Observability infrastructure part of provisioning
"Provisioning" -d-> "CI Runners": How to provision CI Runners
"Provisioning" -d-> "Cell Architecture and tooling": What to provision
"Observability" -d-> "Provisioning": Cell needs to exist to observe
"Configuration Management" -d-> "Provisioning": Cell needs to exist to configure

```

```

@enduml

```

Stakeholders

We have several teams partaking in the operations of Cell.

The first distinction is between teams implementing and maintaining the tools, and teams using those tools.

Areas		Features	
Owners			
----- ----- ----- -----		----- ----- ----- -----	
Integration with Dedicated tools			
		Integration with Release Managers'	
workflows	team::Delivery-Deployments		
		Deployment mechanics using Instrumentor	
and AMP	team::Foundations		
		Cell application reference architectures	
and overlays	team::Ops		
		Cell bootstrapping, tooling, and	
supporting infrastructure	team::Ops		
		Cell deprovisioning	
team::Ops			
Control Plane for cluster state			
		Investigate GitOps model	
team::Delivery-Deployments			
		Investigate CRD + operator	
team::Delivery-Deployments			
Ring-based deployment automation			
		Propagating changes inside a ring	
perimeter	team::Delivery-Deployments		
		Orchestrating changes propagation outside	
ring perimeter	team::Foundations		
		Emergency brake: stopping a package	
rollout	team::Delivery-Deployments		
Rollback capabilities			
		Rollback with downtime (for QA Cell in	
ring 0)	team::Delivery-Deployments		
		Delayed Post Deploy Migrations for	
rollback support	team::Environment Automation		
Observability			
		Cell health metric	
team::Scalability-Observability			
		Fleet health metric	
team::Scalability-Observability			
		Package States	

team::Delivery-Deployments		
Incident Lifecycle Management		
		Paging Engineer On Call
team::Ops		
		Incident tooling
team::Ops		
Network Edge		
		Web Application Firewall
team::Foundations		
		CDN
team::Foundations		
		Load Balancing and networking
team::Foundations		
		Rate Limiting
team::Foundations		

> \ These items may require contributions from various stakeholders in SaaS Platforms and Core Platform. Stakeholders should closely collaborate on this work to ensure appropriate alignment to meet the needs of the owning team and customer teams.

>

> \ These items are for consideration after the Cell 2.0 iteration.

The users of those features are the Release Managers, the Engineer On Call, and the Team:Ops. The following list defines the tasks those groups can perform in the cell cluster:

1. Release Managers

- Command deployments inside the perimeter
- Declare "graduated" packages
- Rollback deployments inside the perimeter

1. Engineer On Call

- Receive alerts for failed deployments
- Can pause a package rollout (not reaching the next ring)
- Drive investigation for failed deployments

1. Team::Ops

- Cells Bootstrapping
 - Provisioning
 - Deprovisioning
 - Re-balancing
- Cell-Ring association

SECTION: engineering/architecture/design-documents/cells/infrastructure/cell_arch_tooling.md

{{< engineering/design-document-header >}}

Summary

This blueprint lays out how cells will be architected, which components will be managed as cell-local or globally, and how a mix of existing and new tooling will be used to provision new cells.

The core of each cell is essentially a Dedicated instance that will be configured to form part of the GitLab.com cluster of cells. This lets us reuse the existing Dedicated tooling (specifically Instrumentor and Amp) for provisioning and configuration of the initial cells. On the other hand, as a consequence of being reliant on Dedicated, many decisions - specifically how a cell is internally architected - are already made for us.

The Foundations team will develop new tooling (provisionally named cellctl) which will be used by callers wanting to interact with cells. This will abstract away the need for the callers to have any knowledge of how a cell is built. It does this by presenting a cell model schema - essentially a reduced version of the Dedicated tenant model schema, with additional cells-related fields added - to clients, which will be translated into a Dedicated tenant model schema that can be used by Dedicated tooling. cellctl, using Terraform, will also be responsible for ensuring that GCP resources are appropriately located within the resource hierarchy. cells-tissue will continue to be the source of truth for cell definitions.

For the initial stages of the cells rollout, we don't believe it is necessary to have a separate provisioning process for cell-local infrastructure, but that option remains on the table in the event of the requirements of cells and Dedicated diverging too greatly to consolidate cleanly within Instrumentor.

Motivation

It has been apparent for some time that we have hit a scalability ceiling on GitLab.com. Noisy neighbor issues abound, and the underlying infrastructure demands an ever-increasing amount of toil from SREs due to the inevitable buildup of unaddressed technical debt under the hood.

To alleviate these problems, we aim to replace the current monolithic architecture with a resilient, horizontally scalable multi-tenanted architecture ("Cells") that provides a high level of isolation between tenants while still providing a largely unchanged experience to the user. This change should also make implementation of customer requirements such as data residency possible.

In doing this, we're also trying to achieve some degree of homogeneity in how GitLab is deployed. Currently, GitLab.com involves myriad different tools to provision: Helm, Chef, Tanka, and Terraform among them. Cells will be using the tooling used to provision GitLab Dedicated tenants, meaning that any improvements to the process for cells can be applied to Dedicated customers too, or vice versa.

Goals

This blueprint will:

- Describe the architecture of each cell
- Enumerate reasons for why certain architectural decisions were made
- Broadly address how existing and new tooling will be used for cell provisioning

Non-Goals

The following is out of scope:

- Details of cell provisioning or configuration management - these will be addressed in separate blueprints (for brevity)
- Process changes (e.g. which team is responsible for what task)
- Low-level implementation details of tooling
- Migration from monolith to cells
 - How to run monolith and cells concurrently

Proposal

- Each cell is effectively its own GitLab Dedicated instance that can interface seamlessly with the rest of GitLab.com, yet is limited to using resources available to that cell only. Users on a cell will not perceive the isolation of the underlying resources
- Dedicated tooling (Instrumentor and Amp) will be used to provision new cells
 - To reduce potential future disruption, common Terraform modules will be split out of the current Instrumentor repo and vendored instead. Foundations will share ownership of the modules with the Dedicated team
 - A separate Amp environment will be setup for cells use
- cellctl is a tool that will be developed by Foundations to translate a cell model schema into a Dedicated tenant model schema, and calling Instrumentor to provision and configure the tenant. Users of cellctl need not be aware of the infrastructure underlying a cell
 - cellctl will also be responsible for ensuring the resource hierarchy in GCP matches the defined hierarchy in cells-tissue

Design and implementation details

Architecture

```
plantuml
@startuml
!include <k8s/Common>
!include <k8s/Context>
!include <k8s/Simplified>
!include <k8s/OSS/all>
!include <gcp/GCPCommon>
!include <gcp/Compute/all>
!include <gcp/Databases/all>
!include <gcp/Security/all>
!include <gcp/Storage/all>

skinparam actorStyle awesome
skinparam frame {
    borderColor<<gcpproject>> 4285F4
}
skinparam cloud {
    borderColor<<gcp>> 34A853
}
```

```

skinparam rectangle {
  BackgroundColor<<blackbox>> black
  FontColor<<blackbox>> white
}

cloud "ClickHouse Cloud" {
  database "ClickHouse" as cell1Clickhouse
  database "ClickHouse" as cell2Clickhouse
}

cloud "Google Cloud Platform" <<gcp>> {

  frame "GitLab Cells" <<gcporganization>> {
    component "Cluster-wide Services" as gs {
      ClusterBoundary(gprd-gke, "gprd-gitlab-gke") {
        NamespaceBoundary(camoproxy, "Camoproxy") {
          KubernetesPod(camopod, "", "")
        }
      }
      ClusterBoundary(ops-gke, "ops-gitlab-gke") {
        NamespaceBoundary(vault, "Vault") {
          KubernetesPod(vaultpod, "", "")
        }
      }
    }
  }

  rectangle "Aggregated Observability Stack" as globalObs <<blackbox>>

  frame "cell-01HV3FBWXHSYBAR2R2" as cell1 <<gcpproject>> {
    ClusterBoundary(cell1gke, "cell1gke") {
      NamespaceBoundary(cell1gitlab, "gitlab") {
        KubernetesPod(cell1web, "Web", "")
        KubernetesPod(cell1shell, "Shell", "")
        KubernetesPod(cell1registry, "Registry", "")
        KubernetesPod(cell1sidekiq, "Sidekiq", "")
      }
      NamespaceBoundary(cell1Observability, "monitoring") {
        KubernetesPod(cell1Prom, "Prometheus", "")
      }
    }

    storage "Storage" as cell1Storage {
      CloudSQL(cell1Postgres, "PostgreSQL", "")
      CloudMemorystore(cell1Redis, "Redis", "")
      CloudStorage(cell1ObjectStorage, "GCS", "")
      KeyManagementService(cell1SecretStorage, "GKMS", "")
      ComputeEngine(cell1Gitaly, "Gitaly", "")
    }

    cell1gke <--> cell1Storage
  }
}

```

```

    cell1gke <---> cell1Clickhouse
}

frame "cell-01HV3FBYX37C18SW05" as cell2 <<gcpproject>> {
  ClusterBoundary(cell2gke, "cell2gke") {
    NamespaceBoundary(cell2gitlab, "gitlab") {
      KubernetesPod(cell2web, "Web", "")
      KubernetesPod(cell2shell, "Shell", "")
      KubernetesPod(cell2registry, "Registry", "")
      KubernetesPod(cell2sidekiq, "Sidekiq", "")
    }
    NamespaceBoundary(cell2Observability, "monitoring") {
      KubernetesPod(cell2Prom, "Prometheus", "")
    }
  }
}

storage "Storage" as cell2Storage {
  CloudSQL(cell2Postgres, "PostgreSQL", "")
  CloudMemorystore(cell2Redis, "Redis", "")
  CloudStorage(cell2ObjectStorage, "GCS", "")
  KeyManagementService(cell2SecretStorage, "GKMS", "")
  ComputeEngine(cell2Gitaly, "Gitaly", "")
}

cell2gke <--> cell2Storage
cell2gke <---> cell2Clickhouse
}

cell1 <---> cell2 blue;line.dotted;text:blue : Private Service Connect
cell1 <-> gs
cell2 <-> gs
cell1Observability <-> globalObs
cell2 <-> globalObs
}
@enduml

```

Components

Each cell contains a Cloud Native Hybrid deployment of GitLab, sized appropriately for its expected workload. This is provided by the GitLab Environment Toolkit (GET) via the Dedicated tool Instrumentor which acts as an orchestration layer on top of GET.

Cell definitions (i.e. size of each cell, which components it includes, etc.) will be stored in the cells-tissue repository, which already contains definitions for a number of cells and rings.

Service	Runs on
-----	-----

Webservice	Kubernetes
Sidekiq	Kubernetes
Supporting services	Kubernetes
Gitaly	Google Compute Engine VM
PostgreSQL	CloudSQL
Redis	Memorystore
Object storage	Google Cloud Storage
Per-cell observability stack	Kubernetes
Inter-cell networking	Private Service Connect
ClickHouse	ClickHouse Cloud

Auxiliary services with a global context - that is, services handling data not specific to any individual cell - will have their lifecycle managed completely independently of cells. Such services include:

- Hashicorp Vault (for storage and management of secrets used in CI, and shared infrastructure secrets such as those used by the observability stack)
 - We will be using the existing production Vault instance which is configured using config-mgmt and deployed in gitlab-helmfiles.
- Camoproxy (for proxying assets)
 - We will be using the existing Camoproxy instance deployed in gitlab-helmfiles.

Naming conventions for cells

The ID field can only be up to 30 characters long due to restrictions imposed by the Dedicated tenant model and GCP project naming.

This limits us in what kind of metadata we can include in the cell name: for example, we can't include the region name.

With this in mind, cells will use a ULID, truncated to 17 characters, lowercased and prefixed with c as its tenant ID. For example, a single cell's ID might be c01hw6tqsamavpgr3n.

A prettier name prefixed with cell- will be used for situations where human readability is preferred, e.g. in the GCP project name. For example, if a cell has the ID of c01hw6tqsamavpgr3n, its prettified name will be cell-c01hw6tqsamavpgr3n.

The length of the ID allows for an additional 7 characters which can be used to name additional GCP projects (should they be needed) related to a single cell. For example, if the cell needs additional projects for Runner or Gitaly resources, they can be named like so, while staying within the 30 character limit:

- cell-c01hw6tqsamavpgr3n-rnr123 or cell-c01hw6tqsamavpgr3n-runner
- cell-c01hw6tqsamavpgr3n-git594 or cell-c01hw6tqsamavpgr3n-gitaly

See this issue for discussion.

Relationship to GCP projects

Each cell will have its own project in GCP to provide greater isolation between cells and simplify management. This is also in accordance with the existing Dedicated tooling, which

assumes one GCP project per tenant.

See this issue for discussion, and this ADR.

Networking

Each cell will have its own VPC, with one subnet per region.

Having a single VPC per cell will make provisioning and management of a cell easier as there will be no need to worry about IP address space overlap issues, as well as make cells secure by default as they will be fully isolated.

Cells will use Private Service Connect to communicate with other cells if required. The definitions specifying how cells are connected will be kept alongside cell definitions that will be used by cellctl to provision the needed resources.

See this issue for discussion, and this ADR.

GCP organization

A new GCP organization (provisionally named GitLab Cells) for the sole purpose of hosting cells will be created. While it will use the same centralized billing account as the existing gitlab.com organization, all resources and permissions under the new org will be controlled authoritatively using Terraform and access will be locked down to only team members who need it. Admin access would be granted in break-glass situations only, and fully auditable.

This was deemed preferable to thoroughly auditing and consolidating the existing organization, which is relied upon by many different teams (so we run the risk of breaking something business-critical) and is only partially managed by Terraform. See this issue for discussion.

Cloud resource hierarchy

The cells tooling (cellctl) will be responsible for creating and maintaining a cloud resource hierarchy under the Gitlab Cells organization. This hierarchy should reflect the filesystem hierarchy of the definitions in the cells-tissue repository.

In this hierarchy, every environment will have its own folder, which in turn contains an Ops folder, and a Cells folder.

Each cell will have its own folder, named for its (prettified) ID, in the Cells folder. This gives us the option of adding new projects for cell-specific components (such as runners or additional Gitaly nodes) to the folder, thus grouping them with the cell concerned, in the future.

The Ops folder will contain the Amp environment GCP project.

```
plantuml
```

```
@startuml
```

```
rectangle "GitLab Cells" <<Organization>> {  
    frame Infrastructure {
```

```

frame "Production" {
  frame "Ops" as ops {
    component "AMP environment Project" as amp
  }

  frame "Cluster-wide Services" as gs {
    component "Topology Service Project" as ts
  }

  frame "Cells" as cells {
    frame "cell-c01hw6tqsamavpgr3n" {
      component "cell-c01hw6tqsamavpgr3n Project" as project1
    }
    frame "cell-c01hw6tqsamg16ksv4" {
      component "cell-c01hw6tqsamg16ksv4 Project" as project2
    }
  }

  project1 -down[hidden]-> project2
  amp -down[hidden]-> ts
}
frame "Sandbox" {
  frame "Ops" as ops2 {
    component "AMP environment Project" as amp2
  }

  frame "Cluster-wide Services" as gs2 {
    component "Topology Service Project" as ts2
  }

  frame "Cells" as cells2 {
    frame "cell-c01hw6tqsam8abagd7" {
      component "cell-c01hw6tqsam8abagd7 Project" as project3
    }
    frame "cell-c01hw6tqsam4khnray" {
      component "cell-c01hw6tqsam4khnray Project" as project4
    }
  }

  project3 -down[hidden]-> project4
  amp2 -down[hidden]-> gs2
}
}
}
@enduml

```

Number of GKE clusters serving a cell

Each cell will run on one GKE cluster. Routing to and between cells will be simpler; the Helm

charts do not support deploying one instance of the application across multiple clusters, while GET does not support managing multiple instances of the application in one cluster.

See this issue for discussion, and this ADR.

Secrets management for cells

We will be using both Google Secret Manager and Hashicorp Vault for secrets management.

Secret Manager will store secrets that are specific to a single cell and thus scoped to a specific GCP project.

Hashicorp Vault will store secrets that are shared across multiple (or all) cells. Such secrets may include credentials for the observability stack.

All Kubernetes secrets will be injected using external-secrets and not provisioned directly.

See this issue for discussion.

Access to cell infrastructure

Much of the underlying infrastructure of a cell is derived from a Dedicated on GCP instance, and as such the procedure will be broadly shared between the two projects.

We will be using GCP's PAM (IAM) functionality to provide access to GCP resources when required.

There will be no access to any GCP resources by default.

There will be 3 entitlements available for elevating privileges at both the project and organization levels:

Name	Requires approval	Purpose
readonly	No	Read-only access to GCP resources. Does not include access to sensitive resources
admin	Yes	Peer-reviewed admin access for SRE access when required for operational work
breakglass	No	Breakglass admin access to resources for SRE EOC during incidents when no approver is available

Elevation of privileges will be logged and audited.

See this issue for discussion.

Observability

In general, the lifecycle of observability components for cells will be owned by the

Observability team.

By default, each Dedicated tenant is provisioned with a fully functional Prometheus/Grafana stack. Cells will reuse this stack, with the intention of aggregating metrics so that queries can be run over multiple cells. More information can be found [here](#).

See this [issue](#) for discussion.

Cell Specific Configuration

A single value in the tenant model will be used to decide if a tenant is going to be a Cell for GitLab.com or a Dedicated Tenant. We'll be using cellid, as shown in ADR: 011.

Tooling

Cells 1.0

In order to achieve the Cells 1.0 milestone in the minimum time possible, Foundations will be doing the following:

- Extracting Terraform modules required for a Dedicated GCP instance out of the Instrumentor monorepo
 - Each module will be split into its own repo and vendored in as an external Terraform module in Instrumentor
 - Thereafter, both the Dedicated team and Foundations will be responsible for maintaining these modules
- Adding reference architectures for cells and their related Jsonnet files to Instrumentor so that appropriate variables can be templated in
- Developing a new tool, provisionally called cellctl, to orchestrate cells
 - For Cells 1.0, cellctl will simply be a thin translation wrapper, transforming its own cell tenant model into an opinionated instance of the tenant model schema used by Dedicated
 - The cells model will be an opinionated derivative of the tenant model schema used by Dedicated and include additional cell-specific fields for use by orchestrators of cellctl.
- Set up a new Amp environment in GCP specifically for provisioning cells

```
plantuml
@startuml
skinparam linetype polyline
skinparam linetype ortho
skinparam frame {
    borderColor<<gcpproject>> 4285F4
}
skinparam cloud {
    borderColor<<gcp>> 34A853
}

folder cells-tissue {
    artifact "Cell Definition" as celldef
}
```

```

component cellctl {
  file "Cell Model Schema" as cellmodel
}

"cells-tissue" -> cellctl
celldef -.> cellmodel

component Instrumentor as instrumentor {
  component tenctl {
    file "Tenant Model Schema" as dedicatedmodel
  }
}

cellctl -> instrumentor

cloud GCP <<gcp>> {
  component Amp as amp

  frame "cell-c01hw6tqsamavpgr3n" as cell <<gcpproject>>

  cellmodel .r.> dedicatedmodel

  instrumentor -> amp
  amp -> cell
}

@enduml

```

Other tooling, such as ringctl, should be able to interface with cellctl to perform operations on one or many cells without needing to be aware of any specifics of how cells are implemented or orchestrated.

Cell tenant model

The tenant model of a cell will follow this structure, split into 2 core fields:

- a metadata struct which contains information used to identify the cell instance (name and labels), in addition to tooling specific information such as which template to use to render the cell definition. Labels can be used to store information relevant to other tools, such as the ring that the cell belongs to.
- a spec struct that contains information about the definition of a cell instance, including the location in which to create the instance.

```

go
// Model entry point
type Cell struct {
  metadata CellMetadata
  spec    CellSpec
}

```

```

// Contains fields specific to Cells
type CellMetadata struct {
    // The identifier of the cell
    name string
    // Optional template specifier
    //
    // If this is not set then the template must be provided
    // externally.
    template string json:",omitempty"
    // Additional annotations
    //
    // To be used for defining features and passing additional
    // context through to the cell template.
    annotations map[string]string json:",omitempty"
    // Additional labels

    // Used to add human understandable context to cell instances.
    // Can be used to perform searches across cell instances.
    labels map[string]string json:",omitempty"
}

// Contains fields related to the definition of the cell through the dedicated tooling
type CellSpec struct {
    // Primary GCP project ID
    projectid string
    // The region we want this cell to be provisioned within
    region string
    // Reference architecture
    referencearchitecture string
    // Instrumentor version
    instrumentorversion string json:",omitempty"
    // GitLab version
    gitlabversion string json:",omitempty"
    // Prerelease version
    prereleaseversion string json:",omitempty"
}

```

cellctl processes these cell instance definitions from YAML files, and translates them through a mapping layer of jsonnet templates to output a tenant model for provisioning through Amp and Instrumentor.

These cell definition files can also be processed by other tooling at the same plane as cellctl to create additional infrastructure with a "global" view of all cell instances.

Beyond Cells 1.0

As cells are gradually built out, we may find that the requirements for Dedicated tenants and cells will diverge, and attempting to consolidate this into Instrumentor may result in unacceptable complexity. This may necessitate an additional layer of resources provisioned and

managed by cells-specific tooling, which would be developed and maintained by Foundations.

Where should something be implemented for cells?

There are four places containing resource definitions for components making up a single cell. Each builds on top of the last:

```
plantuml
@startuml
frame cells-tissue {
    frame Instrumentor {
        frame GET {
            rectangle "Official distribution (Helm chart/Omnibus)"
        }
    }
}
@enduml
```

As we build out cells, there will inevitably be questions about the best place to add something. The following chart should help guide the decision:

```
mermaid
graph TD
    A[I want to add something to all cells] --> B[Should it be part of the official GitLab distribution?]
    B -->|Yes| C[Request to add it to the Helm charts/Omnibus]
    B -->|No| D[Could it be considered best practice for self-administering a GitLab instance?]
    D -->|Yes| E[Request to add it to GET]
    D -->|No| F[Is there current or anticipated future demand from Dedicated customers for it?]
    F -->|Yes| G[Add it to the Instrumentor shared Terraform modules]
    F -->|No| H[Could it benefit Dedicated customers in some way, without too much additional work?]
    H -->|Yes| G
    H -->|No| I[Add it to cells-tissue]
```

SECTION: engineering/architecture/design-documents/cells/infrastructure/deployments.md

{{< engineering/design-document-header >}}

Disclaimer: This blueprint requires more cross-functional alignment - Confidence Level: Low

This blueprint describes a deployment strategy that can support the new scaling dimension introduced by the Cell Architecture.

The complexity of this transition will require many teams in the Platforms section to participate by taking ownership of the features necessary to reach the production grade rating on this architecture.

Introduction

Preamble

From a high level perspective, a Cell Cluster is a system made of only 3 items:

1. Router - An HA routing system deployed independently from the GitLab application.
1. Legacy Cell - The existing GitLab.com infrastructure.
1. Zero or more Cells - GitLab installations authoritative for a limited number of Organizations. Those Cells are deployed using GitLab Dedicated tools.
1. Topology Service - An HA system used for Cluster-wide uniqueness and classification of resources.

```
plantuml
@startuml
```

```
actor User
```

```
cloud router {
component Router as R
}
```

```
component "Legacy Cell" as Legacy
component "Topology Service" as TopologyService
collections "Cells" as Cells
```

```
User ==> R
R ==> Legacy
R ==> Cells
```

```
Cells --> TopologyService
Legacy --> TopologyService
R --> TopologyService
```

```
@enduml
```

As we can see from the diagram, users interact with the system through the router only. Cells communicate with the Topology Service and have a local copy of all the database rows necessary to operate.

It is important to note that even if Cell supports GitLab Geo out of the box, we will not be able to provide this feature to our users until the Router supports it.

Key Terms

- Deployment - The GitLab application and its components being installed into infrastructure
- auto-deploy version - The active version that creates a package viable for deployment
- ring - A logical partition of the cell cluster. In order to deploy to the next ring a package must be validated inside the current ring.

- perimeter - the ring marking the "definition of done" for Release Managers, a package validated inside the perimeter is allowed to rollout in the rest of the fleet
 - graduated version - The version deemed safe to deploy to cells outside of the perimeter
 - .com - Our old existing or currently running infrastructure
 - Legacy Cell - The existing GitLab.com infrastructure.
 - Cell(s) - GitLab installation(s) authoritative for a limited number of Organizations.
- Deployed using GitLab Dedicated tools.

Ring deployment

The scale of the Cell project deployment together with the strong user partitioning maps well with a ring deployment approach.

```
plantuml
@startuml
```

```
skinparam frame {
    borderColor<<perimeter>> red
}
```

left to right direction

```
frame "Ring 3" as r3 {
    component "Cell4" as c4
    component "Cell5" as c5
    component "Cell6" as c6
    component "Cell7" as c7
    component "Cell8" as c8
    component "Cell9" as c9

    frame "Ring 2" as r2 {
        component "Cell1" as c1
        component "Cell2" as c2
        component "Cell3" as c3

        frame "Ring 1" <<perimeter>> as r1 {
            frame "Ring 0" as r0 {
                component "Canary stage" <<legacy>> as cny
                component "QA Cell" as QA

                note as ring0note
                    Ring 0 goes in parallel with canary
                    QA tests executed on canary and QA Cell
                end note
            }
        }
    }
}
```

```
component "Main stage\nLegacy Cell" <<legacy>> as Legacy
```

```
note as perimeternote
The perimeter marks the definition of done for an autodeploy package.
```

```

    When post-deployment migrations are executed inside the perimeter,
    the package is ready to be pulled by the outer rings
    outside of the release managers coordinator pipeline
    end note
}

note as bakingareas
    A package cannot rollout to the next ring before it is successfully
    installed inside the current ring.
end note
}
}

@enduml

```

In the image above we are showing a possible ring layout with a cluster made of the Legacy Cell and 10 Cells, the upper bound of the Cell 1.0 milestone.

The general rule is that:

1. The deployment process progresses from Ring 0 to the outer rings.
1. Rings are a collection of Cells sharing the same risk factor associated to a deployment.
1. Deployments can get halted at any stage and the package will not reach the outer rings.
1. We define the "perimeter" ring that marks the "definition of done" for the Release Managers.
 - Crossing perimeter is the logical point in time of a given package lifecycle after the PDM has successfully run on the Main Stage. Effectively, between Ring 1 and Ring 2 as described throughout this document.
 - A successful run of the Post Deploy Migrations inside the perimeter marks a package as graduated.
 - A graduated package is a valid candidate for the monthly release.
 - A graduated package is rolled out to the rest of the rings automatically.
 - Deployments must be automated: inside the perimeter are responsibility of Release Managers, outside of it are responsibility of Team:Ops.

Application changes lifecycle

In this section, we describe how new packages are deployed in the new infrastructure, explaining the interactions and hooks with the existing tooling and processes.

GitLab.com deployment process is described in great detail in the handbook, we are going to make some simplifications here to reduce the cognitive load to what is necessary to talk about ring deployments.

Today's process

Today GitLab.com deployments consist of two main processes:

1. The release-tools coordinator pipeline for auto-deploy packages is responsible for sequencing deployments and QA through staging and production environments. The timeline of this

process depends on the available release managers, but it usually happens ten times a day.

1. The Post Deployment Migrations pipeline executes the post-deployment migrations of the current package running in production, completing the quality assurance of the deployed changes. This process is usually executed only once a day because it marks a point where it is no longer possible to rollback an environment.

mermaid

flowchart TB

```
subgraph releasetools [Release-tools coordinator pipeline]
    direction LR
    gstgcny[GSTG\n Canary] --> gstgcnyqa[QA]
    gstgcnyqa --> gprdcny[GPRD\n Canary]
    gprdcny --> gprdcnyqa[QA]
    gprdcnyqa --> bakingtime[1hr\n baking time]
    bakingtime --> promotion[.\n manual\npromotion]
    promotion --> gstg[GSTG\nMain] --> gstgqa[QA]
    promotion --> |delay 30m|gprd[GPRD\nMain]
end
pkg((Auto-deploy\npackage\ntagging)) --> releasetools
subgraph pdm [Post Deployment Migrations]
    direction LR
    pdmgstg[GSTG] --> pdmgstgqa[QA] --> pdmgprd[GPRD]
end
end
%% invisible link to help content layout
releasetools ~~~ pdm
```

A package can be released to self-managed customers only after running its post deployment migrations.

Co-existence of the legacy infrastructure and Cells

For some time the existing (legacy) deployment and Cells will co-exist. Initially our ring deployment implementation will only consist of three rings:

- Ring 0: hosting a QA Cell and eventually other experimental builds, this ring is kept in sync with the legacy production canary stage.
- Ring 1: an empty placeholder for the legacy production main stage. As we will be able to begin extracting existing users from the legacy deployment, this ring could host ourselves, our free users, and more in general every organization interested in having a fast track to our new features.
- Ring 2: hosting our first cell for customers.

Because of the nature of our rel